

Artificial Intelligence Laboratory

Transformer Models and Local LLMs

Samarth Bhatnagar

October 1, 2026

Contents

1	Introduction	2
2	Transformer Architecture	2
2.1	Encoder	2
2.2	Decoder	2
2.3	Encoder-Only Models	3
2.4	Decoder-Only Models	3
3	Experimental Setup	3
4	Encoder-Decoder Architecture	4
4.1	BERT2BERT Machine Translation	4
4.2	T5 Translation	4
5	English-to-Hinglish Generation	4
6	Decoder-Only Transformer	5
7	Encoder-Only Transformer	5
8	Comparison of Transformer Architectures	6
9	Running LLMs with Ollama	6
9.1	Ollama Installation	6
9.2	Downloading and Testing Mistral	7
9.3	LangChain Integration	7
10	Comparison with Other LLMs	9
11	Observations	9
12	Conclusion	10

1 Introduction

Transformers are neural network architectures that use attention mechanisms to process sequences. The original Transformer architecture was introduced for machine translation, where one sequence is provided as input and another sequence is generated as output.

Transformers are now used in several different forms. Encoder-only architectures are commonly used for understanding and representing existing text, decoder-only architectures are widely used for autoregressive text generation, and encoder-decoder architectures are used for sequence-to-sequence tasks such as machine translation.

This laboratory investigates these three types of Transformer architectures using the Hugging Face Transformers library. The experiments include machine translation, Hinglish generation, text generation using GPT-2, and sentiment analysis using a BERT-based encoder model.

The laboratory also introduces the use of Ollama for running a GPT-type language model locally or through a Google Colab environment.

2 Transformer Architecture

The Transformer architecture consists of an encoder and decoder in its original form. The encoder processes the input sequence and produces representations that are used by the decoder to generate the output sequence.

The main concepts investigated in this laboratory are:

- Attention mechanism
- Self-attention
- Cross-attention
- Multi-head attention
- Positional encoding
- Masked attention

2.1 Encoder

The encoder receives the input sequence after tokenization and embedding. Positional information is incorporated because the self-attention mechanism itself does not inherently represent the order of tokens.

Each encoder layer contains multi-head self-attention followed by a feed-forward network, together with normalization and residual connections.

2.2 Decoder

The decoder generates the target sequence. In the original encoder-decoder Transformer, each decoder layer contains masked self-attention, cross-attention over the encoder representations, and a feed-forward network.

Masked self-attention prevents the decoder from accessing future tokens during autoregressive generation.

2.3 Encoder-Only Models

Encoder-only Transformers, such as BERT, are primarily used for understanding existing text rather than generating long sequences.

Typical applications include:

- Sentiment analysis
- Semantic search
- Document retrieval
- Text representation
- Clustering

2.4 Decoder-Only Models

Decoder-only Transformers generate text autoregressively. At each step, the model predicts the next token based on the previously generated context.

GPT-style models are examples of decoder-only Transformers and are used for applications such as text generation, chatbots, code generation, question answering, and content generation.

3 Experimental Setup

The experiments were implemented using Python and the Hugging Face Transformers library.

The main library installation used in the notebook was:

```
1 !pip install "transformers[torch]"
```

Listing 1: Installing the Transformers library

The experiments were performed using pre-trained Transformer models available through the Hugging Face model hub.

The following types of models were investigated:

- BERT2BERT encoder-decoder model
- T5 encoder-decoder model
- English-to-Hinglish Flan-T5 model
- GPT-2 decoder-only model
- DistilBERT sentiment-classification model

4 Encoder-Decoder Architecture

4.1 BERT2BERT Machine Translation

The first experiment used the pre-trained model

```
google/bert2bert_L-24_wmt_en_de
```

The input sentence was:

```
Plants create energy through a process known as
```

The model generated the German translation:

```
Pflanzen erzeugen Energie durch einen so genannten Prozess.
```

The experiment demonstrates the use of an encoder-decoder Transformer for sequence-to-sequence translation. The encoder processes the input sentence and the decoder generates the translated sequence.

4.2 T5 Translation

The second encoder-decoder experiment used:

```
google-t5/t5-base
```

The input was given using an instruction-style prefix:

```
translate English to French:
```

```
Let's go have some wine, cause thats how we shine
```

The generated output was:

```
Allons en déguster un peu de vin, car c'est ainsi que nous brillons
```

This experiment demonstrates how T5 treats different NLP tasks as text-to-text problems. The task itself is specified as part of the input text.

5 English-to-Hinglish Generation

The next experiment used the pre-trained model:

```
rvv-karma/English2Hinglish-Flan-T5-Base
```

The input sentence was:

```
life is crazy. Not me
```

The generated output was:

```
Life crazy haiNot me
```

The experiment demonstrates how a sequence-to-sequence Transformer can be fine-tuned for a specific language transformation task.

6 Decoder-Only Transformer

The decoder-only experiment used GPT-2 through the Hugging Face `pipeline` interface.

The model was prompted with:

The future of AI is

The generated output was:

The future of AI is not yet clear, but it is likely to be worth exploring the potential benefits of artificial intelligence and how it might shape the world.

The experiment demonstrates autoregressive text generation. The model generates tokens sequentially based on the preceding context.

Conceptually, the process can be represented as

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1}).$$

Thus, each new token is predicted using the previously available tokens.

7 Encoder-Only Transformer

The encoder-only experiment used a sentiment-analysis pipeline based on a pre-trained DistilBERT model.

The input sentence was:

I'd want to kick you and ensure that you are hurt

The model returned the sentiment label:

NEGATIVE

with a confidence score of approximately

0.9742.

The result demonstrates the use of an encoder-only Transformer for text classification. Instead of generating a new sequence, the model produces a classification label for the input text.

8 Comparison of Transformer Architectures

The experiments demonstrate different uses of Transformer architectures.

Architecture	Model	Experiment
Encoder-Decoder	BERT2BERT	English-to-German translation
Encoder-Decoder	T5	English-to-French translation
Encoder-Decoder	Flan-T5	English-to-Hinglish transformation
Decoder-Only	GPT-2	Autoregressive text generation
Encoder-Only	DistilBERT	Sentiment classification

Table 1: Transformer architectures investigated in the laboratory.

The experiments demonstrate that the same general Transformer framework can support very different applications depending on its architecture, training objective, and pre-trained weights.

9 Running LLMs with Ollama

The second notebook investigated running a language model using Ollama in the Google Colab environment.

9.1 Ollama Installation

The Colab environment initially did not contain the `zstd` dependency required by the current Ollama installation package. The dependency was therefore installed first:

```
1 !apt-get update -qq
2 !apt-get install -y zstd
```

Listing 2: Installing the `zstd` dependency

Ollama was then installed using its installation script:

```
1 !curl -fsSL https://ollama.com/install.sh | sh
```

Listing 3: Installing Ollama

The installation completed successfully. The installation output also indicated that the Ollama API was available at `127.0.0.1:11434`.

Since `systemd` is not normally running in the Google Colab environment, the Ollama server was started manually:

```
1 import subprocess
2 import time
3
4 ollama_process = subprocess.Popen(
5     ["ollama", "serve"],
```

```

6     stdout=subprocess.PIPE,
7     stderr=subprocess.STDOUT,
8     text=True
9 )
10
11 time.sleep(5)
12
13 print("Ollama server started.")

```

Listing 4: Starting the Ollama server

The server was then checked through its local API:

```

1 !curl -s http://localhost:11434/api/tags

```

Listing 5: Checking the Ollama server

9.2 Downloading and Testing Mistral

The Mistral model was downloaded using:

```

1 !ollama pull mistral

```

Listing 6: Downloading the Mistral model

The installed models were checked using:

```

1 !ollama list

```

Listing 7: Listing installed Ollama models

The model was then tested directly using the Ollama command-line interface:

```

1 !ollama run mistral "Who is Sir Isaac Newton?"

```

Listing 8: Testing Mistral directly

This confirmed that the Mistral model could be accessed through the running Ollama server.

9.3 LangChain Integration

After verifying that Ollama was working, the LangChain integration was installed:

```

1 %pip install -U langchain-ollama

```

Listing 9: Installing LangChain Ollama integration

The required classes were imported:

```

1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_ollama.llms import OllamaLLM

```

Listing 10: Importing LangChain components

A prompt template was defined as follows:

```

1 template = """Question: {question}
2
3 Answer: Let's think step by step."""

```

Listing 11: Defining the prompt template

The prompt and Mistral model were then connected into a LangChain pipeline:

```

1 prompt = ChatPromptTemplate.from_template(template)
2
3 model = OllamaLLM(model="mistral")
4
5 chain = prompt | model

```

Listing 12: Creating the LangChain pipeline

The chain was tested using the question:

```

1 question = "Who is Sir Isaac Newton?"
2
3 response = chain.invoke({
4     "question": question
5 })
6
7 print(response)

```

Listing 13: Invoking the Mistral model through LangChain

The notebook was subsequently extended to evaluate Mistral on several different prompts.

```

1 prompts = [
2     "Who is Sir Isaac Newton?",
3     "Explain what a Transformer neural network is.",
4     "Write a Python function to calculate the factorial of a
5     number.",
6     "Why is the sky blue?",
7     "Explain the difference between supervised and unsupervised
8     learning."
9 ]

```

Listing 14: Prompts used for Mistral evaluation

Each prompt was passed through the same LangChain pipeline and the responses were stored for inspection.

```

1 ollama_results = {}
2
3 for question in prompts:
4     response = chain.invoke({"question": question})
5     ollama_results[question] = response
6
7     print("=" * 80)
8     print("QUESTION:")
9     print(question)
10    print()

```

```
11 print("MISTRAL RESPONSE:")
12 print(response)
13 print()
```

Listing 15: Evaluating Mistral on multiple prompts

This experiment demonstrated how a locally hosted language model can be accessed through both the Ollama interface and LangChain.

10 Comparison with Other LLMs

The laboratory instructions propose comparing the responses generated by the locally hosted Mistral model with responses from other large language models such as GPT or Claude.

After successfully configuring Ollama, Mistral was evaluated using the same set of prompts covering factual questions, Transformer architectures, programming, general science, and machine learning.

The following criteria can be used when comparing responses from different language models:

- Correctness
- Relevance
- Quality of reasoning
- Quality of generated code
- Completeness
- Presence of factual errors or hallucinations

A meaningful comparison requires the same prompts to be supplied to each model. The responses can then be examined qualitatively according to the criteria above.

No overall ranking of the models is made without performing the same experiments and examining the corresponding outputs.

11 Observations

Several observations were made during the experiments.

First, encoder-decoder Transformers are well suited to sequence-to-sequence tasks. The BERT2BERT and T5 experiments both successfully transformed English input into another language.

Second, the T5 experiment demonstrated the text-to-text formulation of NLP tasks, where the desired operation can be specified using a textual prefix such as:

translate English to French:

Third, the English-to-Hinglish experiment demonstrated that a sequence-to-sequence Transformer can be specialized for a particular language transformation task using appropriate pre-trained weights.

Fourth, GPT-2 demonstrated autoregressive generation. Instead of producing a fixed classification label, the decoder-only model continued the input sequence by predicting additional tokens.

Fifth, the DistilBERT experiment demonstrated the use of an encoder-only model for classification. The model assigned a negative sentiment label with a score of approximately 0.9742 for the given input.

Finally, the Ollama experiment demonstrated how a language model can be run within the Colab environment using a locally available Ollama server. After installing the required dependency, Ollama was installed, the server was started manually, the Mistral model was downloaded, and the model was accessed both directly and through LangChain.

These experiments show that the Transformer architecture can support substantially different NLP tasks depending on the model architecture, training objective, and pre-trained weights.

12 Conclusion

This laboratory provided hands-on experience with different Transformer architectures using pre-trained models from the Hugging Face Transformers library.

The encoder-decoder experiments demonstrated sequence-to-sequence generation through machine translation and language transformation. The decoder-only experiment demonstrated autoregressive text generation using GPT-2, while the encoder-only experiment demonstrated sentiment classification using DistilBERT.

The experiments also reinforced the importance of attention-based Transformer components such as self-attention, cross-attention, multi-head attention, positional encoding, and masked attention.

The laboratory additionally provided hands-on experience with running a language model using Ollama. The Mistral model was successfully installed and accessed through the Ollama server, and its integration with LangChain was demonstrated using several prompts.

Overall, the laboratory demonstrated that Transformer models form a general architecture that can support substantially different NLP tasks depending on how the model is structured and trained. The Ollama experiment further demonstrated how a language model can be deployed locally and integrated into a Python application through LangChain.